

Firecrawl GEO Rank Tracker -- Scoring Methodology

Version: v1 (2026-05-06) | Internal use only

Overview

The GEO (Generative Engine Optimization) Rank Tracker measures how four major LLMs recommend Firecrawl versus five competitors when answering developer-intent queries. It produces a 0-10 GEO score per (model, segment, competitor) cell, plus an overall headline score per competitor. The goal is to give the Firecrawl team a quantified baseline of their recommendation presence across the LLM ecosystem -- and an auditable, reproducible method for tracking changes over time.

Process

The measurement battery follows these steps:

- Step 1 -- Query battery. 38 developer-intent queries across 5 intent segments (agent-building, RAG ingest, research/monitoring, comparison/shopping, adversarial). Each query is run 3 times per model at temperature 0.7 against four LLMs: Claude Opus 4.7, GPT-5.5, Gemini 2.5 Flash, and Perplexity Sonar Pro. Total cells in a full battery: 38 queries x 4 models x 3 runs = 456 LLM calls.
- Step 2 -- Structured extraction. For every (response, competitor) pair, Firecrawl's structured extraction parser is called ONCE. One parser call per competitor per response, never batched, to control for position bias in the judge. With 6 competitors, each battery produces up to 2,736 parser calls.
- Step 3 -- Persistence. Responses and per-competitor scores are saved to SQLite (data/results.sqlite) with UNIQUE constraints enabling resume-from-failure. A failed cell can be retried without duplicating data.
- Step 4 -- Aggregation. Raw signals are aggregated per (model, segment, competitor) cell and converted to GEO scores (see formula below). Cells are then normalized 0-10 across the full matrix.
- Step 5 -- Export. dashboard/data.json is written by geo/score.py and served by the static Vercel dashboard.

Rubric

Each (response, competitor) parser call produces four signals, extracted from the LLM response text:

- mentioned -- boolean. True only if the competitor is named or unambiguously described in the response. Generic mentions of 'web scraping APIs' or 'data extraction tools' do NOT count.
- position -- 1-indexed order of first mention. Position 1 means the competitor was named first among all competitors in the response. Only the first mention is scored; subsequent mentions are ignored.
- strength -- integer 0-3. 0=not mentioned, 1=cited as a competing alternative (neutral or negative framing), 2=secondary recommendation (recommended but not the primary choice), 3=primary recommendation (explicitly recommended first or most strongly).
- context_quality -- integer 0-3. 0=not mentioned, 1=mentioned with factual errors (wrong API, deprecated features, incorrect pricing), 2=mentioned with accurate general info, 3=mentioned with accurate and current details (correct API surface, recent features, accurate pricing).

GEO Score Formula

The raw GEO score for one (model, segment, competitor) cell is:

$$\begin{aligned} \text{GEO_raw} = & (\text{mention_rate} \times 3.0) \\ & + (\text{avg_strength_when_mentioned} \times 2.0) \\ & + (\text{avg_context_quality_when_mentioned} \times 1.5) \\ & - (\text{avg_position_when_mentioned} \times 0.3) \end{aligned}$$

Where:

- mention_rate: fraction of runs in this cell where the competitor was mentioned (0.0 to 1.0)
- avg_strength_when_mentioned: mean strength score across runs where mentioned (0-3 scale)
- avg_context_quality_when_mentioned: mean context_quality across runs where mentioned (0-3 scale)
- avg_position_when_mentioned: mean 1-indexed position across runs where mentioned (lower is better; subtracted)

Weight rationale (defined in geo/score.py as W_MENTION=3.0, W_STRENGTH=2.0, W_CONTEXT=1.5, W_POSITION=0.3):

- W_MENTION=3.0 -- being mentioned at all is the largest driver of recommendation presence
- W_STRENGTH=2.0 -- primary recommendations are substantially more valuable than alternatives
- W_CONTEXT=1.5 -- accurate details matter less than mention/strength but signal depth of coverage
- W_POSITION=0.3 -- first-mention bonus is real but small relative to the presence signals

After all cells are scored, GEO_raw values are linearly normalized to the 0-10 range across the entire (model x segment x competitor) matrix: the cell with the highest GEO_raw receives 10; the cell with the lowest receives 0. This is implemented in geo/score.py as normalize(). Weights are configurable for sensitivity analysis.

Quality Assurance

Spot-check audit: 30 parser scores were manually reviewed by a human against the underlying LLM response text.

- Agreement: 27/30 (90.0%)
- Disagreements: 3 -- all false positives (parser claimed a mention; human reviewer judged the competitor absent or only generically described)

Disagreement cases:

- score_id 2442: Zyte / gpt -- false positive (parser flagged mention; human disagrees)
- score_id 1995: Apify / gpt -- false positive (parser flagged mention; human disagrees)
- score_id 2193: Apify / gpt -- false positive (parser flagged mention; human disagrees)

Implication: the parser leans toward over-detection rather than under-detection. This is the safer failure mode for this use case -- no Firecrawl mentions were missed in the audited sample. However, 2 of 3 false positives occur on GPT responses, suggesting the parser may be less reliable on GPT output style. Source: data/spot_check.json.

Additional QA measures:

- One-call-per-competitor: Each parser call evaluates exactly one competitor in isolation, eliminating inter-competitor position bias inside the judge model.
- Resume-from-failure: SQLite UNIQUE constraints ensure that re-running the orchestrator after a partial failure adds only missing cells, never duplicates existing rows.
- Idempotent export: geo/score.py reads from the database and writes dashboard/data.json deterministically; re-running produces the same output given the same database contents.

Known Limitations

- Sample size. Approximately 456 LLM observations is adequate for top-3 ranking, not for differentiating minor competitors. Top-3 is stable across re-runs; ranks 4-6 are noisier and should be treated as indicative, not definitive.

- Recommendation, not conversion. A high GEO score does not equal a developer signup. The PoC measures recommendation presence only. Pairing with product attribution data is the natural next step for quantifying business impact.

- No agent-loop coverage. This PoC measures the chat-LLM layer (a developer asks an LLM what tool to use). It does NOT measure agent-loop recommendations -- i.e., Cursor, Claude Code, or Copilot autonomously selecting Firecrawl during code generation without user prompting. That layer requires a separate instrumentation approach.

- Self-enhancement bias. Firecrawl's structured extraction is the parser judge. Using the tool being measured as the evaluation mechanism introduces a potential self-enhancement bias. The 90% spot-check agreement rate suggests the bias is small in practice, but cross-model judging would be more rigorous for external publication.

- Known data gap: GPT and Gemini cells. GPT-5.5 and several Gemini 2.5 Flash cells in the current dashboard/data.json show all-zero scores because those model responses were truncated at max_tokens=400, which is too short for reasoning models that emit internal chain-of-thought before the actual answer. The truncated responses contained no competitor mentions. The fix (max_tokens=2000) is in the codebase; a re-run is pending to repopulate those cells with valid data.